TASK 1

Linux cheat-sheet: -

File System Navigation

pwd — Print Working Directory. Shows the full path of the directory you're currently in.

cd — Change Directory.

- cd /path/to/dir → go to a specific directory
- cd .. → move up one level
- cd ~ or cd (no argument) → go to home directory
- cd - → go to the previous directory you were in

ls — List. Shows files/folders in the current (or specified) directory.

- ls -l → long format (permissions, owner, size, date)
- ls -a → show hidden files (those starting with .)
- ls -la → combine both
- ls -lh → long format with human-readable sizes (KB, MB, GB)
- ls -lt → sort by most recently modified

File & Directory Permissions

chmod — Change Mode. Modifies read/write/execute permissions.

- Numeric: chmod 755 file (owner=rwx, group=rx, others=rx)
- Symbolic: chmod u+x file (add execute for owner), chmod g-w file (remove write for group), chmod o=r file (set others to read-only)
- chmod -R 755 folder/ → apply recursively to all contents

chown — Change Owner. Changes which user/group owns a file.

- chown user file → change owner only
- chown user:group file → change owner and group
- chown -R user:group folder/ → apply recursively

Package Management

apt — High-level package manager (Debian/Ubuntu).

- apt update → refresh the list of available package versions (doesn't install anything)
- apt upgrade → install available updates for installed packages
- apt install <package> → install a package and its dependencies

- apt remove <package> → uninstall a package, keep its config files
- apt purge <package> → uninstall a package and delete its config files
- apt list --installed → show all installed packages

dpkg — Low-level package tool that apt is built on.

- dpkg -i package.deb → install a local .deb file directly
- dpkg -l → list all installed packages
- dpkg -r <package> → remove a package
- dpkg -L <package> → list all files that a package installed

Networking Commands

ifconfig (legacy) / **ip addr** (modern replacement) — Shows network interfaces and their assigned IP addresses, status (up/down), MAC address.

ping — Sends ICMP echo requests to a host to check if it's reachable and measure round-trip latency.

- ping google.com
- ping -c 4 google.com → send only 4 packets, then stop

netstat (legacy) / **ss** (modern replacement) — Shows active network connections, listening ports, and routing info.

- netstat -t → TCP connections
- netstat -u → UDP connections
- netstat -l → listening sockets only
- netstat -n → show numeric addresses (skip DNS resolution)
- netstat -p → show the process/PID using each connection
- Combined: netstat -tulnp → all listening TCP/UDP ports with numeric addresses and owning process

traceroute — Shows the hop-by-hop path packets take to reach a destination, with latency at each hop.

- traceroute example.com

Wireshark (Packet Capture)

Capture filters (applied before capture, limits what's recorded):

- host 192.168.1.10 → only traffic to/from that IP
- port 80 → only traffic on port 80
- net 192.168.1.0/24 → only traffic within that subnet

Display filters (applied after capture, filters what's shown):

- ip.addr == 192.168.1.10 → traffic to/from an IP
- tcp.port == 443 → filter by port
- http → show only HTTP traffic
- http.request.method == "POST" → only POST requests
- tcp.flags.syn == 1 && tcp.flags.ack == 0 → show only SYN packets (useful for spotting a SYN flood)
- dns → show only DNS traffic
- Follow → TCP Stream (right-click a packet) → reconstructs the full conversation for that connection

CLI equivalent — tshark:

- tshark -i eth0 # capture on interface eth0
- tshark -i eth0 -f "port 80" # with a capture filter
- tshark -r capture.pcap # read from a saved capture file
- tshark -r capture.pcap -Y "http" # apply a display filter to a saved file

Nmap (Network Scanning)

- nmap 192.168.1.1 # basic scan, default top 1000 ports
- nmap -sV 192.168.1.1 # service/version detection on open ports
- nmap -sS 192.168.1.1 # SYN scan ("stealth" scan, doesn't complete TCP handshake)
- nmap -sU 192.168.1.1 # UDP scan
- nmap -p- 192.168.1.1 # scan all 65535 ports
- nmap -p 22,80,443 192.168.1.1 # scan specific ports
- nmap -O 192.168.1.1 # OS detection
- nmap -A 192.168.1.1 # aggressive: OS + version + scripts + traceroute
- nmap -sC 192.168.1.1 # run default NSE (scripting engine) scripts
- nmap 192.168.1.0/24 # scan an entire subnet
- nmap -oN output.txt 192.168.1.1 # save output to a file

Burp Suite (Web Proxy)

Burp is GUI-driven rather than command-based, but the core workflow commands/settings to know:

- **Proxy → Intercept on/off** — toggles whether requests are paused for inspection/modification before forwarding
- **Browser proxy setup**: configure browser to route traffic through `127.0.0.1:8080` (Burp's default listener) so it passes through Burp
- **Send to Repeater** (`Ctrl+R`) — resend and manually edit/replay a single request repeatedly
- **Send to Intruder** (`Ctrl+I`) — automate payload injection (fuzzing) into a request, e.g., for brute-forcing a parameter
- **Target → Scope** — restrict which hosts Burp actively processes/attacks
- **Decoder** — encode/decode strings (Base64, URL encoding, hex) found in requests
- **Comparer** — diff two requests/responses side by side

Netcat (Network Debugging)

- nc -l -p 4444 # listen on port 4444 (server mode)
- nc 192.168.1.10 4444 # connect to a host on a port (client mode)
- nc -v 192.168.1.10 80 # verbose connection (shows success/failure)
- nc -zv 192.168.1.10 20-80 # scan a port range for open ports (-z = no data sent)
- nc -l -p 4444 > file.txt # receive a file
- nc 192.168.1.10 4444 < file.txt # send a file
- nc -e /bin/bash -l -p 4444 # bind shell (spawns a shell on connection — classic backdoor technique, know it defensively)
- nc -lvp 4444 # set up a listener to catch a reverse shell